Hermes: The Complete Free-API Handbook

Set up Hermes, connect Nous Portal Free and OpenRouter, combine legitimate allowances through FreeLLMAPI, add optional local models, and keep paid use explicit.

Independent practical setup handbook. Sources/catalogs checked **20 October 2026**. This covers the provider families actually configured in the author's stack plus additional eligible upstreams—not just OpenRouter. Catalogs, pricing, account eligibility, and tool APIs can change. No secret/account database is included and no unlimited-free guarantee is made.

Access labels matter: zero-priced route ≠ recurring allowance ≠ trial credits ≠ earned credits ≠ subscription-included usage ≠ discounted paid inference. Each provider chapter identifies the difference.

The PDF has clickable links and selectable text. Use the Markdown version for exact copy/paste commands. Illustrated cards are a quick-start overview; this handbook contains the full setup details.

Contents

• 0. What Hermes is, what you need, and what to set up first

• 1. The strategy

• 2. Install Hermes

• 3. Nous Portal: free models through the official Hermes provider

• 4. Connect OpenRouter and choose a free model

• 5. FreeLLMAPI: combine your legitimate free allowances behind one local API

• 6. Other provider families actually configured here

• 7. More free allowances you can add directly or through FreeLLMAPI

• 8. Use the same APIs in your own scripts and other applications

• 9. Configure free-only fallbacks and aliases

• 10. Add optional local models

• 11. Keep background work and auxiliary calls free

• 12. Install the routing policy

• 13. Use paid models deliberately

• 14. Optional Jev decision gates

• 15. Verify the setup

• 16. Troubleshooting and privacy

• 17. Links and maintenance

• 18. NanoGPT (paid-last, crypto, no KYC)

• 19. PPQ (paid-last, invite-only, privacy-first)

• 20. CheaperInference (discounted paid aggregator)

• 21. Kimi (cheap paid + referral credits)

0. What Hermes is, what you need, and what to set up first

Hermes Agent is the application. Nous Portal, OpenRouter, and other services are model/API providers. FreeLLMAPI is an optional router in front of providers. Ollama/llama.cpp are local inference engines. These are separate layers; installing one does not automatically create accounts or free allowances at the others.

```
YOU → Hermes Agent → Nous free model
                   → OpenRouter free model
                   → FreeLLMAPI → enabled/eligible upstream provider
                   → local model
                   → explicitly authorized paid model (optional)

OTHER APPS/SCRIPTS → the same supported APIs with their own approved credentials
```

What Hermes can do

With a suitable tool-capable model and approved tools, Hermes can read/write files, run commands, research the web, work with repositories, keep reusable skills and memory, delegate work, schedule jobs, and connect to messaging platforms. It also offers optional browser, voice, vision, and media tools.

Free software does not mean every tool backend is free. A free text model can still call a paid search, image, browser, memory, transcription, or cloud-compute service if you configure one. Review those separately.

Minimum practical setup

1. Install Hermes from its official distribution.

2. Connect **one** real free route—Nous Portal Free or a verified OpenRouter free model.

3. Verify a harmless real tool call.

4. Set free-only automatic fallbacks and pin background/auxiliary work.

5. Add FreeLLMAPI and more legitimate allowances only when you need them.

6. Add local inference if you want offline/private model work or a separate quota-independent backup.

The provider chapters are alternative/optional connection paths, not a sequence of commands to run blindly. Choose the primary route you actually want (Nous, OpenRouter, an audited gateway, or local); provider-specific examples later in the book should be adapted to that choice. You do not need accounts at every provider, a GPU, Docker, a swarm, a paid subscription, or every plugin just to begin.

Each additional service adds limits, data policies, failure modes, and maintenance.

What the author's installation actually contains

The redacted inventory found Nous as the main provider, explicit OpenRouter free fallbacks, an active FreeLLMAPI service, Pollinations/NanoGPT/Cheaper Inference custom-provider families, and a reachable local Ollama instance. Several aliases point at the same provider family; they are not separate sources of free capacity.

The full inventory and evidence limits are in `docs/PROVIDER-STATUS.md` in the repository. This guide includes those families and useful additional FreeLLMAPI upstreams. It does not claim every registered adapter has a valid key, remaining allowance, or tested completion.

Two important findings: current NanoGPT pricing puts its free model on the web only; current Hugging Face hosted Inference Providers gives free accounts no included monthly credits. Do not put either API into a strict-free chain based only on an old list.

Keep a simple provider worksheet

For each route, record only non-secret facts:

Field	Example
Provider and protocol	Nous Portal / OpenAI-compatible chat completions
Base URL	Its official endpoint, not a copied guess
Model ID	Exact verified free entry
Access type	Zero-price model, recurring allowance, trial, earned credits, subscription, or paid
Limits/reset	Current account-specific ceilings and reset time
Data policy	Whether this workload may leave the machine and how data can be used
Verification	Catalog only, authenticated completion, tool call, or failure test

Never record API-key values in a shared worksheet, screenshot, repo, or guide.

Learn these Hermes controls first

```
hermes                # Start the terminal interface
hermes desktop        # Start the desktop interface when installed/supported
hermes model          # Choose/authenticate a model provider
hermes tools          # Inspect and choose tool backends
hermes skills list    # Inspect installed reusable workflows
hermes profile list   # Inspect available profiles
hermes cron list      # Inspect scheduled jobs and pins
hermes config check   # Check configuration issues
hermes doctor         # Diagnose installation/runtime issues
```

Inside a chat, `/model` opens model selection; named aliases can provide shortcuts. A new default does not necessarily replace the model in an already-open conversation. Read the model/provider

display rather than assuming a change applied everywhere.

CLI documentation: https://hermes-agent.nousresearch.com/docs/user-guide/cli.

Work safely on real files

Start in an empty test directory or disposable repository. Ask for a read-only task first, then a tiny edit, then verify the file yourself through real tool output. Keep command approvals enabled; a less capable free model should not receive broader permissions to compensate for weak reasoning. Use version control/backups before allowing important edits or destructive operations.

Security guide: https://hermes-agent.nousresearch.com/docs/user-guide/security.

Skills, memory, and project instructions

• **Skills** package reusable procedures and scripts. Load only relevant ones; a giant pile of unrelated instructions increases prompt size and can hurt smaller models.

• **Memory** stores ongoing facts/preferences; do not use it as a secret manager.

• **Project instructions** such as `AGENTS.md`/`.hermes.md` apply within their documented scope. Merge the free-first policy into existing instructions instead of overwriting them.

• **Profiles** separate settings/workflows, but are not a hard OS security sandbox. Some authentication can be shared across profiles; review the actual secret/provider behavior before claiming strong separation.

References: https://hermes-agent.nousresearch.com/docs/user-guide/features/skills, https://hermes-agent.nousresearch.com/docs/user-guide/features/memory, https://hermes-agent.nousresearch.com/docs/user-guide/features/context-files, and https://hermes-agent.nousresearch.com/docs/user-guide/profiles.

Web, browser, speech, and media without accidental bills

Use a local browser/backend when it fits. Where available, a local/self-hosted search backend, local speech recognition, or local speech synthesis can avoid a hosted API bill. They still have installation/runtime costs and network behavior to review. Do not assume an account at Nous, Pollinations, or another aggregator grants every media tool for free.

Inspect `hermes tools` and provider terms before enabling a backend. For example, “free speech output” from an online service is not offline/private speech. Prefer an actual local backend when the data must remain local.

Tool guide: https://hermes-agent.nousresearch.com/docs/user-guide/features/tools. Browser guide: https://hermes-agent.nousresearch.com/docs/user-guide/features/browser. Web-search guide: https://hermes-agent.nousresearch.com/docs/user-guide/features/web-search.

Telegram and other messaging platforms

Get ordinary local chat working first. Then follow the official Telegram guide: https://hermes-agent.nousresearch.com/docs/user-guide/messaging/telegram.

Create your own bot through BotFather, enter its token only in the trusted local gateway setup, configure the allowed-user/chat controls, and test the bot from the intended account. Keep the gateway

running for messaging and agent-backed scheduled work. Do not publish bot tokens or grant unknown people access to a machine-controlling agent.

Other messaging options: https://hermes-agent.nousresearch.com/docs/user-guide/messaging. Remote OAuth/browser callbacks can need SSH forwarding or the documented remote-login flow; do not expose login callbacks casually to the internet.

Two setup routes in this repository

Human-led: follow this handbook's account/setup chapters, then validate the routes and spending policy.

AI-assisted: give the portable `hermes-free-first-setup` skill to an agent. It must inventory providers, distinguish free/allowance/trial/paid access, guide secure account connection, and obtain approval before changes. The supplied script configures publicly verifiable Nous or OpenRouter free defaults; it does not automatically create accounts, read keys, install every provider, or prove a custom FreeLLMAPI chain is free.

Do not let an AI force everything onto OpenRouter if you chose Nous or already have a working gateway. Prefer the verified route you actually want, and configure other routes alongside it rather than silently replacing it.

1. The strategy

Use this ladder:

```
Can an ordinary command or script do it reliably?
  YES → run the command/script. No LLM needed.
  NO  → does a suitable local model handle it acceptably?
          YES → local model.
          NO  → capable free cloud model.
                  ↓ service failure
                another explicitly free route, or wait/local/stop.
                  ↓ task still fails objective checks
                ask before paid inference.
```

The free cloud model can still be your everyday default. “Local first” does not mean forcing every request through a weak or slow local model before allowing a better free model. Use local for private material and small, repeatable jobs; use free cloud directly for demanding agent work.

Three different kinds of “free”

Route	What it actually means	Trade-off
Local model	No hosted inference bill; runs on your hardware	Electricity, hardware, storage, and speed still matter
Free cloud model	Provider currently lists the route at zero price	Quotas, availability, and third-party data handling
Subscription/OAuth model	Included in an existing plan or allowance	Not universally free; subscription and usage limits still exist

An open-weight model is not automatically free to access through somebody else's API. A free browser playground is not necessarily a free API. Promotional credits are not permanent free service.

Pick the spending mode

Recommended: strict approval. Paid models run only when you explicitly select or authorize one. Exhausted free quota means wait, go local, or ask—not silently spend.

Optional: preauthorized last resort. You may later authorize a bounded escalation mechanism for a specific task. That requires enforcement outside an ordinary prompt: allowed models, provider-side spending limits, validation, and an audit trail. A confidence score alone is not spending permission.

These are separate mechanisms:

• **Fallback:** switches routes after an API/service error.

• **Quality escalation:** moves to a stronger model after a result fails tests or other objective checks.

Hermes' fallback chain is not a judge of whether an otherwise successful answer was correct.

2. Install Hermes

Official project: https://github.com/NousResearch/hermes-agent

Installation instructions: https://hermes-agent.nousresearch.com/docs/getting-started/installation

Desktop

Download from https://hermes-agent.nousresearch.com/ and follow the instructions for your platform. You do not need an expensive GPU for free cloud models.

Linux, supported macOS, or Windows through WSL2

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

This executes downloaded code. Review the installer first if that does not match your security policy. Do not run the installer as root merely for convenience. Follow the installer's shell/PATH instructions or reopen your terminal.

Then:

```
hermes --version
hermes setup
hermes doctor
```

Native Windows and Termux have separate installation paths in the official docs. The shell examples below target Bash on Linux/macOS/WSL; use the desktop UI where shell syntax differs.

Existing installation? Back up first

Find your active files:

```
hermes config path
hermes config env-path
```

On a POSIX shell:

```
HERMES_DATA="${HERMES_HOME:-$HOME/.hermes}"
cp -p "$HERMES_DATA/config.yaml" "$HERMES_DATA/config.yaml.before-free-first"
```

Keep that backup private: older configurations might contain credentials. Do not share your whole Hermes directory, `.env`, `auth.json`, logs, or session database.

These changes affect the active profile. Other profiles, remote gateways, and existing pinned jobs have their own settings; audit them separately.

3. Nous Portal: free models through the official Hermes provider

What you can get free

Nous Portal has a real **Free plan**. The official plan page lists **$0, free models only, and $0 monthly credits**: https://portal.nousresearch.com/manage-subscription.

That means access to eligible zero-priced routes—not a free balance for Claude/GPT or every other paid model. The Nous-hosted search/image/browser/speech Tool Gateway is a separate entitlement. Do not enable or rely on paid hosted tools merely because the chat model is free.

Public API documentation: https://portal.nousresearch.com/api-docs.

OpenAI-compatible inference base URL:

```
https://inference-api.nousresearch.com/v1
```

Create an account and connect Hermes

1. Open https://portal.nousresearch.com/ and create/sign into your own account.

2. Inspect the plan page and choose **Free** if you want no subscription charge. Do not select Plus/Super/Ultra just to follow this guide.

3. In Hermes, run `hermes model`, choose **Nous Portal**, and finish the browser/device login.

4. Select an exact currently free, tool-capable model—not just any model with “Hermes” in its name.

5. Decline paid Tool Gateway opt-ins unless you deliberately want that separate service and understand its billing.

6. Start a new chat and verify the actual provider/model metadata.

The convenient `hermes setup --portal` onboarding also configures/offers hosted tools; it is not a command that magically makes every Portal model and tool free. Review every selection. Official integration guide: https://hermes-agent.nousresearch.com/docs/integrations/nous-portal.

For the main route used in the author's current configuration:

```
hermes config set model.provider nous
hermes config set model.default 'poolside/laguna-s-2.1:free'
hermes config set model.base_url 'https://inference-api.nousresearch.com/v1'
```

Authenticate using the provider picker first. Do not paste tokens into those commands or change only the model while leaving a different provider's endpoint/credential overrides attached.

Free entries observed in the public catalog

The following exact IDs had explicit zero pricing and advertised tools at the check:

- `poolside/laguna-s-2.1:free`
- `poolside/laguna-xs-2.1:free`
- `upstage/solar-mini4:free`
- `meituan/longcat-2.0:free`
- `stepfun/step-3.7-flash:free`
- `meituan/longcat-2.5-preview:free`
- `reflection/beam-1.0-501b-a23b:free`

This is a catalog observation, not a benchmark or a guarantee your account can immediately call every entry. Model retirement and capacity restrictions still apply. Step 3.7 Flash and LongCat 2.5 Preview advertised image input; most other listed routes were text-only. Test actual vision/tool behavior before depending on it.

Public catalog: https://inference-api.nousresearch.com/v1/models.

A historical configured fallback can outlive a model. In this inventory, `stepfun/step-5-preview:free` was configured but absent from the current free catalog. Do not copy stale routes blindly.

Reuse the API in your own applications

If your Portal dashboard permits API-key creation on your chosen free plan, generate a key there and use the inference base URL above with an exact free model. An OAuth login inside Hermes is not itself an API key you should copy into unrelated tools. Keep both credentials private.

Fresh-account free-plan API-key issuance was not tested here. The general API docs discuss credits/subscriptions, while the plan page explicitly offers free models. If the dashboard requests a payment before issuing a key, stop rather than spending just to follow this guide; check the current account entitlements/support or use the supported OAuth route inside Hermes and another verified free API for external apps.

The API docs currently list Free limits of **50 requests/minute and 500,000 tokens/minute**. Those are rate ceilings, not a grant of unlimited monthly tokens. Check account-specific limits and eligibility. Unauthenticated generation can return payment requirements; that does not turn x402/USDC into a free route. This guide does not enable payment protocols.

Pin other work deliberately

If Nous is your chosen free provider, select free Nous routes for delegation, cron, and auxiliary slots too. For example, the cron fleet defaults are:

```
hermes config set cron.model_provider nous
hermes config set cron.model 'poolside/laguna-s-2.1:free'
```

Existing pinned jobs are still separate. For vision, choose a currently advertised image-capable free route rather than assigning a text-only model. Keep fallback chains explicitly free/local; authenticate a

separate OpenRouter backup before adding it.

4. Connect OpenRouter and choose a free model

Create your own key

1. Open https://openrouter.ai/ and create an account.

2. Create a key at https://openrouter.ai/settings/keys.

3. In a real terminal, run `hermes model` and choose **OpenRouter**, or add the key through Hermes' provider/key settings.

4. Enter the key only in the trusted local setup UI—not in an AI chat, screenshot, shared config, or public repository.

5. Choose a free model from the picker, or use its exact model ID.

Hermes normally stores API credentials in its `.env` credential file; settings go in `config.yaml`. External secret-manager integration is optional: https://hermes-agent.nousresearch.com/docs/user-guide/secrets.

A generic browser password vault does not automatically wire a provider key into Hermes. Use the provider setup flow or a documented secret-source integration.

Suggested starting model

The following model was listed with zero prompt/completion price and tool support in the live OpenRouter catalog when this guide was checked:

```
nvidia/nemotron-3-ultra-550b-a55b:free
```

Model page: https://openrouter.ai/nvidia/nemotron-3-ultra-550b-a55b:free

After authenticating through the wizard, these commands make the selection explicit:

```
hermes config set model.provider openrouter
hermes config set model.default 'nvidia/nemotron-3-ultra-550b-a55b:free'
hermes config set model.base_url 'https://openrouter.ai/api/v1'
```

Open a **new chat** afterward. An existing chat may retain its earlier model until you switch it explicitly.

This is a starting choice, not a permanent “best model” claim or a successful inference benchmark. If it disappears, use the discovery steps below.

Prefer not to choose a specific model?

Use **OpenRouter's Free Models Router**:

```
openrouter/free
```

Docs: https://openrouter.ai/docs/guides/routing/routers/free-router

It filters for request capabilities, then randomly selects from eligible free models. It is not a guaranteed best-model selector. It can be convenient for starting out; a fixed model is easier to evaluate and troubleshoot.

Do not confuse `openrouter/free` with `openrouter/auto`. The auto router is not a free-only guarantee.

Find currently free, tool-capable models

Browse https://openrouter.ai/models?pricing=free.

For a machine-readable check, run this in a terminal with Python 3. It queries the public catalog without an API key and makes no inference requests:

```
python3 - <<'PY'
import json
from decimal import Decimal, InvalidOperation
from urllib.request import urlopen

with urlopen('https://openrouter.ai/api/v1/models', timeout=30) as response:
    models = json.load(response)['data']

for model in sorted(models, key=lambda item: item['id']):
    pricing = model.get('pricing', {})
    if not {'prompt', 'completion'} <= pricing.keys():
        continue
    try:
        free = all(Decimal(str(value)) == 0 for value in pricing.values())
    except (InvalidOperation, ValueError, TypeError):
        continue
    tools = 'tools' in model.get('supported_parameters', [])
    if free and tools:
        print(model['id'], 'context:', model.get('context_length', '?'))
PY
```

Checking every listed price avoids assuming that zero text-token prices imply zero price for every other listed component. Catalog metadata still does not establish availability, reliable tool use, or zero cost for separately enabled plugins/tools.

Other entries observed in the checked catalog included:

- `poolside/laguna-s-2.1:free`
- `nvidia/nemotron-3-super-120b-a12b:free`
- `google/gemma-4-31b-it:free`
- `minimax/minimax-m3.1-flash:free`

Do not append `:free` to an arbitrary paid model and assume it works. The exact free variant must exist in the catalog.

Understand the quotas before you build automations

At verification time, OpenRouter's published free-variant limits were:

All-time credits purchased	Requests/minute	Requests/day
Less than 10 credits	20	50
At least 10 credits	20	1,000

Check your actual account at https://openrouter.ai/docs/api/reference/limits; exceptions and policy changes can apply. A credit purchase is **optional and costs money**—it is not necessary to describe the basic setup as free. More keys/accounts do not bypass the shared limits.

One agent task can consume many requests: tool loops, compression, retries, and subagents all add traffic. Fifty requests does not mean fifty substantial coding tasks.

5. FreeLLMAPI: combine your legitimate free allowances behind one local API

What this is—and what it is not

FreeLLMAPI is a **self-hosted router**, not a company handing you unlimited model credits. You bring provider keys/allowances, enable suitable routes, and it presents an OpenAI-compatible endpoint with routing and fallback.

Use the project actually installed in this setup:

• Source: https://github.com/tashfeenahmed/freellmapi

• Install guide: https://github.com/tashfeenahmed/freellmapi/blob/main/docs/en/install/01-install.md

• Releases: https://github.com/tashfeenahmed/freellmapi/releases/latest

• API guide: https://github.com/tashfeenahmed/freellmapi/blob/main/docs/en/api/01-rest-api.md

There are similarly named forks, including an unrelated PHP implementation. Do not mix their installation or API instructions.

The router is free/open source. The project's **optional paid Premium live catalog** is distinct: its README describes a delayed monthly catalog snapshot for free installs and a current feed for paying users. You do not need to buy Premium to run the router. Aggregate “billions of tokens” headlines are theoretical sums across providers—not an entitlement for every new user, a tested capacity on this machine, or a production SLA.

Choose one installation path

Desktop: use the official release for your supported platform. Open its dashboard from the tray. Inspect the listening address/port; desktop releases can use a different port from the server default.

Docker/server: install Docker/Compose through the official instructions for your OS first, then review the project's installer. On Linux/macOS/WSL, the documented shortcut is:

```
curl -fsSL https://freellmapi.co/install.sh | bash
```

This executes remote code and requires Docker. Review it before running. It is optional; manual Docker/source installation is documented upstream. Keep the default loopback-only binding. Do not overwrite an existing FreeLLMAPI `.env`, encryption key, or database while “reinstalling.”

Source: check the repository's current Node/npm requirements, install/build as documented, and run the built server. This inventory's package declares Node `>=20.18.0 <25.0.0` and npm `>=10.0.0`. A system service is preferable to leaving an important gateway attached to a temporary terminal session. The author's installation uses a user service; other operating systems should use their

supported supervisor or the desktop app.

The usual server dashboard is http://127.0.0.1:3001/ and the usual model API base is:

```
http://127.0.0.1:3001/v1
```

These are local addresses on the machine running the gateway. On a phone, another PC, or inside a container, `localhost` means that device/container—not your desktop. Use a secured deployment/tunnel when remote access is genuinely needed; do not casually expose the unified key or publish a public proxy.

First-run setup and the three different secrets

1. Open your local dashboard. Server installs have their supported first-run administrator setup; the desktop app has a different local sign-in flow.

2. Complete any setup-code/password prompts locally. Do not paste passwords or setup codes into an AI chat.

3. Open **Keys** and add your upstream providers one at a time. Use your own legitimate accounts and the correct provider IDs.

4. Copy the gateway's **unified API key** from its trusted local UI into the application that will call the gateway.

5. Configure/test one model before enabling a wide automatic chain.

Do not confuse:

Secret	Purpose	Goes where
Upstream provider key	Authenticates Groq/Gemini/OpenRouter/etc.	FreeLLMAPI's local Keys UI/store
Unified API key	Authenticates your applications to FreeLLMAPI	Hermes custom-provider/key settings or your app's secret source
Encryption key	Protects the local encrypted credential store	Gateway deployment secret/config, backed up privately

The administrator password/session is another credential, not the model API key. Never retrieve the unified key by dumping the database into chat. Use the supported dashboard/credential flow. A new installation can generate a different unified key; an old one then fails authentication even when the URL is correct.

Add useful free upstreams

Start with one or two that you can genuinely access, for example a Groq free account, eligible Gemini free-tier API, Mistral Free mode, or exact OpenRouter `:free` models. See the provider-by-provider signup instructions below. Verify regional eligibility, limits, data terms, and whether the account is actually on a free plan.

• Add the key in the matching **Keys** entry.

• Enable only appropriate models in **Models**.

• Test a short completion and structured tool call in the gateway playground.

• Inspect **Analytics** and the served-provider/route metadata.

• Keep experimental/trial and paid/credit-spending lanes separated.

Adding a key does not make every model at that provider free. An expired trial is not a transient error to retry forever. Do not create extra accounts or rotate keys to evade provider terms or quota limits.

Build a genuinely free-only chain

The gateway's own routing policy matters as much as Hermes' fallback list. Review enabled models, named chains, custom providers, routing strategies, and model-specific capabilities. Disable paid routes and unintended external extras. Do not include subscription/PAYG NanoGPT or Cheaper Inference just because they are installed elsewhere in Hermes.

The model ID `auto` is FreeLLMAPI's router and is only as safe as its enabled upstream chain. It is **not** the same service as `openrouter/auto`, and is not an independent proof of zero cost. Optional decision-model routing can itself invoke another service; keep it off until its pricing and limits are verified.

The free snapshot catalog can be stale. Reconcile it with current first-party terms—especially services whose free offerings changed, such as Hugging Face hosted inference. Do not buy a catalog subscription automatically to “fix” model availability.

Connect Hermes to FreeLLMAPI

In `hermes model`, choose a **Custom Endpoint** or add a named custom provider through provider settings:

```
Provider name: freellmapi
Base URL:      http://127.0.0.1:3001/v1
API key:       your local gateway's unified key, entered only in the trusted key UI
Model:         a tested fixed model, or auto after auditing the gateway's chain
```

Discover models if the supported UI offers it. Do not paste a key into `config.yaml` or into a command-line argument visible to other processes. A manually named provider must exist before aliases like `freellmapi/auto` can work.

A settings-only provider shape to recognize is:

```
providers:
  freellmapi:
    name: FreeLLMAPI
    base_url: http://127.0.0.1:3001/v1
    model: auto
    key_env: HERMES_FREELLMAPI_API_KEY
    discover_models: true
```

`HERMES_FREELLMAPI_API_KEY` is an example credential-variable name, not a secret. Your actual configured name may differ. Add the credential through the supported local setup/secret-source flow so that it matches the provider's `key_env`. The snippet is not a whole-file replacement or permission to bypass config protections.

Add Nous as a gateway upstream only when explicitly supported

The installed FreeLLMAPI source did **not** register a native `nous` adapter. Keep Nous directly in Hermes, or use a supported **Custom OpenAI-compatible upstream** with the Nous inference URL, your own eligible Portal API key, and exact verified free model IDs. Do not give a generic OpenRouter adapter a Nous key or invent a `nous` platform ID.

Verify the gateway without leaking a key

Use its trusted playground/client or a credential-aware local application. The expected checks are:

• Gateway is running on the intended interface/port.

• Authentication succeeds using the unified key—not an admin-session token.

• The selected upstream actually completes a request.

• The structured tool call works, and Analytics identifies a compliant served route.

• Under a controlled outage, only approved free/local backups are selected.

An unauthenticated `/v1/models` request returning **401** proves the protected server is responding, not that the unified key is invalid. That is the non-secret reachability check performed here. No unified key, credential database, or authenticated upstream routing was inspected for this guide.

The project describes itself as personal experimentation/learning software. Respect that and each provider's terms. For an important production service, free-tier availability and this router do not supply a production guarantee.

6. Other provider families actually configured here

These are part of the stack inventory, but not all are free API providers.

Pollinations: real API use funded by Quest Pollen

• Home: https://pollinations.ai/

• Keys: https://enter.pollinations.ai/keys

• Quests: https://enter.pollinations.ai/quests

• API documentation: https://gen.pollinations.ai/docs

• OpenAI-compatible base: `https://gen.pollinations.ai/v1`

1. Create/sign into your account and inspect the wallet/available Quests.

2. Complete legitimate offered Quests or other supported free-credit activities. Rewards and eligibility can change; no fixed free balance is promised here.

3. Create a secret API key with restricted models, expiry, and a spending budget.

4. Add a Hermes custom provider with the base URL above and a currently eligible model from its live catalog. The configured example on this machine was `openai/gpt-5.4-nano`; that is a priced route funded by Pollen, not automatically a zero-price model.

5. Verify the wallet debit and remaining free/earned balance before batch use. Stop when the approved allowance ends; do not enable automatic top-ups.

The current docs use canonical `publisher/model` names. Some old short aliases still work, but use the live catalog rather than assuming legacy `openai` aliases are permanent. Keep secret keys out of browser frontend code. For apps used by other people, use the documented **Connect User Wallets/BYOP** authorization so each user knowingly authorizes their own scoped spending. A publishable app key identifies the app; it is not an unlimited free secret key.

NanoGPT: distinguish the web free model from paid API allowances

• Pricing: https://nano-gpt.com/pricing

• API information: https://nano-gpt.com/api

• Base URL used here: `https://nano-gpt.com/api/v1`

The public pricing page currently says **one free model on the web only**. API access is PAYG or included with the paid Pro subscription. Pro is advertised at **$12/month** with **60 million included input tokens per week**, subject to its current included-model and usage rules.

Therefore the configured NanoGPT routes in this stack are **not evidence of a generally free API**. If you already pay for a plan, an included route may cost no additional amount until the allowance is exhausted, but the subscription itself is paid. Confirm the exact model's inclusion and disable spending past the cap where the provider offers that setting.

Several local provider names/aliases can point to this same company. Do not count them as separate free providers or encourage multi-account quota evasion. Keep this family out of strict zero-cost automatic fallbacks unless a specific account/model has a verified zero-cost entitlement and enforceable stop condition.

Cheaper Inference: discounted paid access, not free API

• Home/pricing: https://cheaperinference.com/

• Base URL: `https://api.cheaperinference.com/v1`

The provider is configured here, but its public page advertises usage-based paid inference with funding from **$5**. No recurring free API entitlement was established. Treat it as an optional paid-last-resort provider only after explicit authorization. A promotion, if separately offered to a particular account, is a finite allowance—not a permanent free tier.

Local Ollama and Hermes-managed llama.cpp

These are genuinely useful no-hosted-inference-bill options. The local Ollama server here responded and listed a customized Qwen coder model plus an embedding model; that observation is not an instruction to download the same custom tag on another machine.

Use the local-model chapter for hardware-fit checks and setup. No online provider key is needed for ordinary loopback-only Ollama/managed local inference. Your GPU vendor does not restrict which hosted APIs you can use: using NVIDIA's hosted API does not require buying an NVIDIA GPU.

Other logged-in subscriptions and tool services

ChatGPT/Codex, xAI/SuperGrok, Copilot, Portal paid plans, and similar subscriptions may provide included usage. They are not universally free API access just because an OAuth login works. A model-provider credential, a browser credential vault, and a tool connector are different integrations. Verify the actual entitlement before enabling them, and keep them out of strict-free unattended chains.

7. More free allowances you can add directly or through FreeLLMAPI

These are useful upstream options—not a claim that every one is connected and working on the author's machine. Create your own account and review first-party terms before adding a key. Availability, country restrictions, models, quotas, and allowed production use differ.

Provider	How to get access	What to call it honestly	Hermes / gateway connection
Groq	Create a console account and API key	Rate-limited free-plan API on eligible models; account/organization limits apply	Custom OpenAI-compatible endpoint `https://api.groq.com/openai/v1`, or FreeLLMAPI `groq`
Google Gemini	Create an API key in Google AI Studio; choose a model with an applicable Free Tier	Eligible free-tier API; free-tier data terms may differ from paid	Hermes Gemini provider, or FreeLLMAPI `google`
Mistral	Create Studio account/key in Free mode, no credit card required per current quickstart	Included free-mode usage within account limits; PAYG is separate	Custom endpoint `https://api.mistral.ai/v1`, or FreeLLMAPI `mistral`
Cerebras	Create a cloud console account/key and inspect Limits	Currently documented Free Trial tier; do not promise permanent unlimited access	Custom endpoint `https://api.cerebras.ai/v1`, or FreeLLMAPI `cerebras`
NVIDIA API Catalog	Sign into build.nvidia.com and generate a model API key	Hosted prototyping/trial credits; not a perpetual zero-price guarantee	Hermes NVIDIA provider, or FreeLLMAPI `nvidia`; use the endpoint from the model page
Cloudflare Workers AI	Create account, obtain account ID and suitably scoped Workers AI API token	Daily free allocation on supported models/plans; paid overage exists on paid plans	Supported Cloudflare adapter/custom endpoint; ensure the installed gateway version supports it
Cohere	Create an evaluation key in its dashboard	Free evaluation/trial usage with limits; production keys are paid	Supported native/compatible adapter; do not assume every FreeLLMAPI build includes it
Hugging Face Inference Providers	Create account/token, then inspect billing	Current free accounts have **no included monthly credits**; subscription credits and purchases are paid	Useful model-download source and paid/included inference route, not a default free API lane

Groq: account → key → one tested model

Open https://console.groq.com/keys and create your own key. Inspect https://console.groq.com/settings/limits and the current model catalog. Put the key in the Groq entry in FreeLLMAPI, or in Hermes' custom-provider key setup with the base URL in the table. Test one currently available tool-capable model; do not assume all models share the same request/token caps.

Official limits: https://console.groq.com/docs/rate-limits. Limits are at organization level, not per key. Upgrading to a paid Developer plan is not part of this free setup.

Gemini: choose the free API tier, not just a free website

Get a key at https://aistudio.google.com/apikey. Review https://ai.google.dev/gemini-api/docs/pricing for the exact model/capability you intend to use and its Free Tier column. Some models/features have no free tier. Read the data-use terms: free API input/output may be used differently from paid service data. Do not send secrets or private business data merely because a key was free to create.

Add the key through Hermes' Gemini setup or the gateway's Google entry. Leave paid billing/upgrades off unless authorized. Check your project limits; a free AI Studio interface is not proof every API endpoint is free.

Mistral: current Free mode setup

Follow https://docs.mistral.ai/getting-started/quickstarts/studio/activate-and-generate-api-key. Create the account at https://console.mistral.ai/, open **API Keys**, name a key, set expiry/scope, and store it privately. The official guide says Free mode requires no credit card. Inspect your organization's included usage and limits rather than repeating old “one billion tokens” claims.

Limits/billing controls: https://docs.mistral.ai/admin/billing-usage/usage-limits. Keep pay-as-you-go disabled if you want usage to stop at the free allowance. Add the key to the gateway's Mistral entry or the compatible endpoint in Hermes, then test only a model available to your account.

Cerebras: inspect the actual Free Trial tier

Start at https://cloud.cerebras.ai/. Create the API key, select an allowed model, and read the console's **Limits** page. The current official table labels the free lane **Free Trial** and publishes request/token ceilings. Do not present it as an unlimited ongoing production service. Add to the matching gateway/custom endpoint and bound output tokens to avoid quota overestimation.

Official limits: https://inference-docs.cerebras.ai/support/rate-limits.

NVIDIA: hosted trials are independent of your local GPU

Start at https://build.nvidia.com/explore/discover, choose a model, and follow **Get API Key**. Review any eligibility and trial-credit balance/expiry. Use the model page's current base URL, model ID, and capability requirements.

Quickstart: https://docs.api.nvidia.com/nim/docs/api-quickstart. FAQ: https://docs.api.nvidia.com/nim/docs/faq. The FAQ distinguishes browser previews from remote API calls that consume trial credits. Self-hosted NIM/enterprise licenses are a different product and are not required to call the hosted API.

Cloudflare: keep the account on a fail-closed free allocation

Create/sign into https://dash.cloudflare.com/. Find **Workers AI**, obtain your account ID, and create an appropriately scoped API token. Use its official compatibility/REST instructions or a supported gateway adapter. Do not invent a normal `/v1` URL without the required account path.

Pricing: https://developers.cloudflare.com/workers-ai/platform/pricing/. The current documented free allocation is **10,000 Neurons/day**. Neurons are compute units, not tokens. The Free plan stops at its limit; paid-plan usage above the allocation can bill. Verify model eligibility and plan details before enabling a route.

Cohere and Hugging Face: special limits, not a free-for-all

Cohere evaluation keys are available at https://dashboard.cohere.com/api-keys; current docs describe evaluation keys as free/limited and production keys as paid. Trial keys are limited to **1,000 API calls/month** in the published rate-limit guide: https://docs.cohere.com/docs/rate-limits. Respect evaluation-use restrictions and check each model.

Hugging Face remains useful for downloading open weights, but its current hosted Inference Providers pricing page lists **no monthly credits for free accounts**: https://huggingface.co/docs/inference-providers/pricing. Do not enable its hosted route in a strict-free chain based on an outdated aggregator catalog. Local use of downloaded models is separate from hosted inference billing.

8. Use the same APIs in your own scripts and other applications

The four settings every compatible client needs

```
API protocol: OpenAI-compatible chat completions (when the provider supports it)
Base URL:     the actual provider or local gateway /v1 endpoint
Model:        an exact, eligible model ID for that endpoint
Credential:   that endpoint's own API key from a trusted local secret source
```

A model ID that works on OpenRouter is not automatically valid on Nous or FreeLLMAPI. An OAuth token from one service is not a general API key for another. A local gateway's unified key is not an upstream provider key.

Route	Base URL	Example selection
Nous Portal	`https://inference-api.nousresearch.com/v1`	`poolside/laguna-s-2.1:free`, with eligible Portal API access
OpenRouter	`https://openrouter.ai/api/v1`	An exact checked `:free` ID or `openrouter/free`
FreeLLMAPI	`http://127.0.0.1:3001/v1` for the usual server install	A tested fixed model, or `auto` after auditing enabled routes
Pollinations	`https://gen.pollinations.ai/v1`	A current eligible model funded by an approved Pollen allowance

Route	Base URL	Example selection
Ollama	`http://127.0.0.1:11434/v1` when using its default port	An exact installed model ID

Credential-safe client example

This Python example uses the OpenAI SDK if your application already uses it. `LLM_BASE_URL`, `LLM_MODEL`, and `LLM_API_KEY` must already be supplied by your trusted local environment/secret integration. Do not put a real key into source code, a command line, or chat. Use an isolated application environment, not Hermes' managed Python installation, if adding SDK dependencies.

```
import os
from openai import OpenAI

client = OpenAI(
    base_url=os.environ['LLM_BASE_URL'],
    api_key=os.environ['LLM_API_KEY'],
)
response = client.chat.completions.create(
    model=os.environ['LLM_MODEL'],
    messages=[{'role': 'user', 'content': 'Reply with exactly: API_ROUTE_OK'}],
    max_tokens=64,
)
print(response.choices[0].message.content)
```

This makes a real generation request and consumes quota/credits on the chosen endpoint. It is an example for users to run after checking eligibility; it was **not** executed with their credentials for this guide. Start with a harmless tiny prompt, inspect provider activity/cost, then test a real tool call before building a larger agent workflow.

For CLI/desktop apps, use their supported provider settings or documented integration. A compatible API does not imply every client supports every transport, vision format, tool schema, reasoning parameter, or authentication flow. FreeLLMAPI's client documentation is at https://github.com/tashfeenahmed/freellmapi/blob/main/docs/en/clients/01-agent-clients.md.

Do not publish a shared server-side key in a frontend or Telegram channel. For applications with multiple users, provide individual authorization/budgets and comply with the upstream terms. A personal proxy and someone else's free account are not a public redistribution license.

9. Configure free-only fallbacks and aliases

Convenient names

These aliases and the sample fallback list are the OpenRouter variant. Keep Nous aliases/backups on `nous` with exact current Nous IDs when that is your chosen route, or use the reviewed custom-provider workflow for a gateway.

```
hermes config set model.aliases.free 'openrouter/nvidia/nemotron-3-ultra-550b-a55b:free'
hermes config set model.aliases.freebackup 'openrouter/poolside/laguna-s-2.1:free'
hermes config set model.aliases.freerouter 'openrouter/openrouter/free'
```

The doubled `openrouter/openrouter/free` is intentional: the first part names the Hermes provider; the remaining `openrouter/free` is the OpenRouter model ID.

Inside a chat:

```
/model free
/model freebackup
/model freerouter
```

Free-only fallback chain

The interactive route is:

```
hermes fallback
```

Choose only verified free routes. Inspect the result:

```
hermes fallback list
```

Or set this explicit starter chain with the CLI:

```
hermes config set fallback_providers '[{"provider":"openrouter","model":"poolside/laguna-s-
    2.1:free"},{"provider":"openrouter","model":"openrouter/free"}]'
```

That command replaces your existing chain. Back it up first and retain any needed local/free entries deliberately. Check any legacy entry as well:

```
hermes config get fallback_providers
hermes config get fallback_model
```

If an old `fallback_model` is unwanted, remove it:

```
hermes config unset fallback_model
```

Never add a paid model "at the end just in case." Automatic error fallback does not wait for a natural-language instruction to approve spending. The same rule applies to an upstream proxy/router's own fallback configuration.

All these cloud fallbacks still share OpenRouter's account-level quota. They help with individual model outages, not an exhausted daily allowance. A local fallback or another legitimately available free provider is a separate route.

Docs: https://hermes-agent.nousresearch.com/docs/user-guide/features/fallback-providers

10. Add optional local models

You can skip this entire section and use free cloud models without local hardware investment.

Easiest: Hermes-managed llama.cpp

1. Open **Settings** → **Providers** → **Local Models**.
2. Click **Install runtime**.
3. Choose a tool-capable model the memory-fit indicator says will fit your machine.
4. Download it and click **Use**.
5. Open a new chat and perform the real tool test in the verification chapter.
6. To keep free cloud as your daily default, select OpenRouter again afterward.

The terminal equivalent begins with `hermes model` → **Local models**.

Docs: https://hermes-agent.nousresearch.com/docs/user-guide/local-models

Hermes manages engine selection, GPU offload, and model fitting. This is the simplest cross-vendor path; do not install CUDA on an AMD system. Do not copy somebody else's VRAM, context, port, or thermal settings blindly.

The managed provider is `llamacpp`. To add a local alias, copy the **exact installed model ID** shown by Hermes:

```
hermes config set model.aliases.local 'llamacpp/EXACT_INSTALLED_MODEL_ID'
```

`EXACT_INSTALLED_MODEL_ID` is a placeholder; replace it before running the command. Then `/model local` selects it. The alias selects a model; it is not by itself proof that the local runtime is configured and running.

You may also add the installed model using `hermes fallback add` → the local provider. Remember: cloud fallback from a local primary sends conversation data off the machine. Leave cloud fallbacks out of an offline/private profile.

Already using Ollama?

Reuse it instead of installing a second inference stack.

• Download: https://ollama.com/download

• Model library: https://ollama.com/library

• Hermes manual guide: https://hermes-agent.nousresearch.com/docs/guides/local-ollama-setup

Check the service:

```
ollama --version
ollama list
curl --fail-with-body http://127.0.0.1:11434/api/tags
```

Use Hermes' **Custom Endpoint** setup with:

```
Base URL: http://127.0.0.1:11434/v1
Model:    the exact model ID from ollama list
API key:  empty or a dummy value if the setup screen requires one
```

The usual Ollama port is `11434`; a particular installation may use something else. Discover the actual endpoint rather than copying a custom port from another person's machine. Do not start a competing server on an occupied port.

A model being good at code generation does not establish reliable structured tool calling. Also check the configured context window: model weights fitting in VRAM does not mean a long agent prompt plus KV cache will fit. Start with a small tool-capable model and measure.

Keep local servers bound to loopback unless you intentionally secure LAN access. An unauthenticated local endpoint should not be exposed to the internet.

11. Keep background work and auxiliary calls free

Your main chat model is not the only possible source of charges.

Audit these places

Component	Free-first rule
Main model	Explicit free or local provider/model
Main fallback chain	Free/local entries only
Compression, titles, vision, approvals, MCP helpers	Inspect assignments and per-task fallback chains
Delegated subagents	Pin free/local or enforce safe inheritance
Cron fleet default	Pin independently from interactive chat
Individual cron jobs	Inspect existing provider/model pins
Other profiles and Kanban workers	Audit their own defaults and per-task overrides
MCP sampling and third-party plugins	Check model overrides, independent billing, and external API costs
Search, images, cloud browsers, speech, memory services	Separate services can cost money even with a free chat model

Subagent default

The commands below are an **OpenRouter-specific example**, not a requirement to replace a working Nous or FreeLLMAPI setup. For Nous, use `provider: nous` and an eligible exact Nous free ID; the Nous chapter and `--provider nous` helper cover that choice. For an audited gateway/local route, configure its actual supported provider/endpoint and independently verify the upstream policy. Do not run examples for an account you have not connected.

A conservative OpenRouter starting point:

```
hermes config set delegation.provider openrouter
hermes config set delegation.model 'nvidia/nemotron-3-ultra-550b-a55b:free'
hermes config set delegation.fallback_providers '[]'
hermes config set delegation.max_concurrent_children 2
```

Here `[]` deliberately disables the delegation fallback chain. You can add tested free-only backups later. Explicit per-call or worker overrides still need auditing. Parallel children consume quota faster, so start small.

Cron fleet default

```
hermes config set cron.model_provider openrouter
hermes config set cron.model 'nvidia/nemotron-3-ultra-550b-a55b:free'
hermes cron list
```

This keeps unpinned cron jobs on the chosen free model even while you manually use a paid model in chat. It does **not** overwrite already pinned jobs.

For one existing job:

```
hermes cron edit JOB_ID --provider openrouter --model 'nvidia/nemotron-3-ultra-550b-a55b:free'
```

Replace `JOB_ID` with the real identifier. A pinned cron job does not use the general model fallback chain under the currently documented behavior; a failed free route should fail rather than spend elsewhere.

For deterministic checks, prefer `--no-agent` script jobs. Example shape, after you have created and tested the script under your active Hermes data directory's `scripts/`:

```
hermes cron create 'every 1h' --name disk-check --script disk_check.py --no-agent --deliver local
```

The script must already exist. Empty stdout is silent; stdout is delivered without an LLM. Do not schedule a model to ask whether a file changed when a file comparison can answer directly. Agent-backed scheduled execution needs the gateway running; see the cron docs for your deployment.

Docs: https://hermes-agent.nousresearch.com/docs/user-guide/features/cron

Auxiliary models

Open the Models settings and expand **auxiliary**. Inspect all slots—not just compression. Current docs say `auto` first uses the configured main route, follows explicitly configured fallback policy, and does not invent another logged-in provider when a main provider is explicitly selected.

But if you manually switch the main chat to a paid model, `auto` may follow it. **Pin the auxiliary slots you want to remain free independently.** Clear stale custom endpoints/credentials when changing routes; the picker handles this more safely than changing only a model string.

Example for compression on a clean slot:

```
hermes config set auxiliary.compression.provider openrouter
hermes config set auxiliary.compression.model 'nvidia/nemotron-3-ultra-550b-a55b:free'
```

Repeat through the UI for titles and other relevant tasks. Vision needs an actually vision-capable route; if none is available free, fail or ask rather than silently use a paid vision model.

Docs: https://hermes-agent.nousresearch.com/docs/user-guide/configuring-models

12. Install the routing policy

Save the following instructions in your applicable global/project instructions or ask Hermes to create a `free-first-routing` skill and attach it to relevant automations. Context-file behavior is documented at https://hermes-agent.nousresearch.com/docs/user-guide/features/context-files.

Merge with existing instructions; do not overwrite your whole `SOUL.md` or `AGENTS.md`. A project file alone will not automatically govern unrelated cron jobs and every other profile.

Copyable strict-approval policy

```
FREE-FIRST MODEL POLICY

1. Prefer deterministic tools/scripts when they solve the task reliably.
2. Use a suitable local model for private data and small repeatable work.
3. Use the configured free cloud model for other work. Local is not a
   mandatory extra hop before a capable free model.
4. Use only verified free/local routes in automatic fallback chains.
5. Verify results with tests, real tool output, or other objective checks.
   Do not equate an HTTP success with a correct answer.
6. On transient failures, honor Retry-After and retry at most twice.
   On exhausted daily quota, wait, go local, or ask. Do not rotate keys
   or accounts to evade limits.
7. After a failed quality check, try one materially different free/local
   approach if useful. Do not loop endlessly through the same failure.
8. Never invoke a paid model unless I explicitly choose or authorize it.
   A router confidence score does not grant permission to spend.
9. Keep cron, subagents, auxiliary models, and external tool APIs within
   the same policy. Never silently send private/local-only data to cloud.
10. Before paid use, name the provider/model, explain the failed free
    alternatives, and state the allowed scope and spending ceiling.
    If cost cannot be estimated, say so.
11. After authorized paid work, return to the free default. Do not change
    global defaults or recurring jobs to paid as a side effect.
12. If no compliant route works, stop with an honest blocker. Never invent
    execution results to make the free route look successful.
```

This is behavioral guidance, not a billing firewall. The stronger controls are free-only model selections, independent background pins, provider-side budgets/model restrictions where supported, and separation from paid credentials.

For the strongest practical spending separation, keep paid access in a different Hermes profile or separate deployment. Merely adding aliases does not restrict what a fully privileged process can do. A profile is organizational separation, not an OS security sandbox; review shared secret sources and use a separate OS user/container if hard isolation matters.

Profiles: https://hermes-agent.nousresearch.com/docs/user-guide/profiles

13. Use paid models deliberately

You do not need to configure a paid provider to complete the free setup.

If you want one later:

1. Connect it through `hermes model` using your own account.

2. Verify its exact model ID, billing terms, and limits.

3. Do not add it to any automatic fallback list.

4. Prefer a separate paid profile, or explicitly select it for a bounded session/task.

5. Return to `/model free` afterward if using the same profile.

An existing ChatGPT, Copilot, or other subscription can offer an included route, but it is still a subscription—not universally free API access. Do not assume any service accepts another service's OAuth token.

Optional alias pattern:

```
hermes config set model.aliases.paidreview 'PROVIDER/EXACT_MODEL_ID'
```

Both parts are placeholders. Configure the provider first and replace them with real supported values. Then select `/model paidreview` deliberately.

A useful authorization looks like:

Use my configured paid reviewer for this one review only. Do not change defaults or cron jobs. Keep total spend within my provider-side budget; if you cannot establish the cost, ask before proceeding.

Avoid promising an exact dollar ceiling based on a prompt alone. Enforce budgets at the provider or an explicitly budget-aware dispatcher. Long-context model switches can trigger a full uncached reread of the conversation.

14. Optional Jev decision gates

Jev supplies typed judgments—yes/no, a choice, or a score. It can help decide:

• whether a failure appears transient from the supplied evidence;

• whether a result passed a stated quality rubric;

• whether to retry, stop, or request human review;

• which allowed free/local route is appropriate.

It does not inspect your machine by magic. Gather logs, test results, and task constraints first, then submit only the evidence needed for the decision.

Cost warning

Jev is optional and its hosted API is not automatically free. A free playground does not make automated API access free. At verification time, the linked independent hosted gateway advertised prepaid API access. Skip it for a strict zero-inference-bill setup unless you have separately verified a free allowance or functioning local backend.

• Official TypeSafe site: https://typesafe.ai/

• Official docs: https://docs.typesafe.ai/

• Community MCP implementation: https://github.com/codaaiteam/jev-mcp

• Independent gateway's MCP guide: https://jevtypesafeai.com/integrations/mcp

• Independent gateway pricing: https://jevtypesafeai.com/pricing

The community repository identifies the linked playground/gateway as independent and not affiliated with TypeSafe AI. Check whom you are trusting with your prompts and keys.

Optional Hermes MCP setup

Use Hermes' MCP settings/add flow. Install Node.js/npm if you do not already have `npx` available. For the linked implementation:

```
Name:    jev
Command: npx
Args:    -y github:codaaiteam/jev-mcp
```

Pass `TYPESAFE_API_KEY` to the server from a local secret source, using a reference rather than writing the key into shareable config. Hermes supports `${TYPESAFE_API_KEY}` references in MCP configuration. The setup's key must match the backend: hosted `jv_live_` keys belong to the independent gateway; official TypeSafe credentials use its official API. Let the documented adapter select the endpoint, or configure the correct one explicitly.

Use the Hermes MCP UI/CLI instead of blindly replacing your YAML. The shape to recognize—not a whole-file replacement—is:

```
mcp_servers:
  jev:
    command: npx
    args: ["-y", "github:codaaiteam/jev-mcp"]
    env:
      TYPESAFE_API_KEY: "${TYPESAFE_API_KEY}"
    sampling:
      enabled: false
```

After trying the documented upstream command, pin a reviewed revision/version for reproducibility instead of silently following future upstream changes. Begin a new Hermes session so the tools are discovered.

Tool names differ across implementations. This repository documents `jev_check`, `jev_classify`, `jev_score`, `jev_gate`, and `jev_decide`; another installation might expose `jev_route`. Use the tools your installation actually lists.

The confidence rule

A conservative policy can require **confidence ≥ 0.90** before taking an already authorized automatic routing action. Treat that as your operating threshold, not proof of 90% accuracy on every task. Test calibration on your own representative cases.

If the tool returns `probability`, determine what proposition it describes. For “can free solve this?”, a low probability is not the same thing as low confidence in the judgment. Do not blindly compare every numeric field against `0.90`.

Paid last resort, safely

The logic should look like this; it is policy pseudocode, **not** an installed dispatcher:

```
if user explicitly chose an available paid model:
    run only the authorized scope, subject to the budget
elif deterministic/local/free route passes the objective checks:
    return that result
elif explicit per-task automatic-paid permission exists
     and all allowed cheaper routes failed the relevant checks
     and a validated gate recommends escalation at confidence >= 0.90
     and the model is on the paid allowlist
     and the dispatcher/provider enforces the remaining budget:
    run the bounded paid step; validate it; record spend and reason
else:
    stop or ask for approval
```

If Jev is unavailable, returns malformed output, or lacks a meaningful confidence value, **fail closed to asking**. Never let the gate manufacture spending authorization. Installing MCP tools alone does not install a cost-enforcing router or automatic model-switch hook.

For most people, the strict manual-approval mode is simpler and sufficient.

15. Verify the setup

Inspect settings

```
hermes --version
hermes config get model
hermes fallback list
hermes config get delegation
hermes config get cron
hermes cron list
hermes config get auxiliary
hermes config check
hermes doctor
```

Inspect locally. Review/redact any output before sharing it, especially from old configs with inline credentials. `config check` helps catch mistakes; it is not a certificate that every feature is free.

Test one real cloud completion

```
hermes chat --provider openrouter \
  --model 'nvidia/nemotron-3-ultra-550b-a55b:free' \
  --oneshot -q 'Reply with exactly: FREE_ROUTE_OK'
```

This consumes quota. Check Hermes' provider/model metadata and your provider activity at https://openrouter.ai/activity. A model claiming its own identity in generated text is not verification.

Test real tool calling

In an empty test directory, ask:

Use a terminal or file tool to list this directory. Report what the tool actually returned. Do not modify any files.

Pass: an actual tool call and corresponding result appear. A prose description or printed JSON pretending to be a tool call does not pass.

Repeat after selecting `/model local`. Confirm in Hermes' model/provider display that it is local. For privacy-sensitive testing, ensure auxiliary calls, fallback chains, and tools are local too; local inference by itself does not make web browsing offline.

Confirm the spending behavior

• All primary and auxiliary fallback entries are free/local.

• Cron fleet default is independent from the current chat model.

• No existing job or worker has an unexpected paid pin.

• Paid provider credentials are absent from the strict-free deployment, or controls are deliberately configured.

• No paid external tools or upstream router policies bypass your intended budget.

• A simulated outage in a disposable profile stops or takes a free/local route; it does not call paid inference.

Do outage tests in a disposable profile, not by damaging your working key. If you do not test failure handling, label it untested.

What was verified for this guide

The expanded edition also inspected the redacted installed-provider families and FreeLLMAPI/local-service reachability, fetched Nous/OpenRouter public catalogs, and retrieved current first-party free/paid terms. See the repository's `docs/PROVIDER-STATUS.md`, `docs/LINK-CHECKS.json`, and verification notes for scope and limitations. No credential database, wallet balance, or authenticated provider completion was inspected for that inventory.

• Official Hermes documentation and OpenRouter free-routing/limit documentation were retrieved.

• The public OpenRouter catalog was queried; suggested entries were checked for zero listed pricing and advertised tool support.

• CLI help was checked on Hermes `v0.22.2+2232.g86fa9e6`.

• Fifteen core configuration commands were executed successfully against a disposable `HERMES_HOME`, including aliases, the JSON fallback list, delegation settings, cron defaults, and compression overrides.

• The resulting config was parsed and key types/values were asserted.

• The exact catalog-discovery example was executed, Bash snippets passed syntax checks, the YAML example parsed, and the original edition’s 31 source links returned HTTP 200. Expanded-edition link checks are recorded in the repository verification notes.

Not performed: fresh OS installation, authenticated completion benchmarks for every suggested model, a new local-model download, a new Jev installation, or a live paid-escalation failure test. Your active configuration was not modified to produce this guide.

16. Troubleshooting and privacy

Symptom	Likely explanation	What to do
HTTP 401	Missing/rejected credential, sometimes wrong endpoint	Reconnect in provider settings; do not print the key
HTTP 402	Credit/key limit, negative balance, or a paid route/component	Inspect error metadata and actual route; do not automatically top up
HTTP 403	Budget, access policy, or a WAF/bot wall	Inspect response type; an HTML challenge is not proof the key is invalid
HTTP 429	Account quota or upstream capacity	Honor `Retry-After`; use local/wait; don't create accounts to evade quotas
HTTP 404/model missing	Retired ID or wrong base URL	Refresh the catalog; select the exact currently listed ID
Free model can't use tools	Missing/weak tool support or server parser issue	Test actual structured tool calls, not chat fluency
Ollama/local connection refused	Server down, wrong host/port, or container-localhost confusion	Verify endpoint/service; inside containers, localhost is the container
Slow local first response	Weight loading or prompt prefill	Use a smaller model, reduce unneeded tool/prompt load, inspect memory fit
Context/memory errors	Prompt/KV cache exceeds the configured window or memory	Choose an appropriate model/context; don't force a huge window blindly
Default changed but current chat did not	Existing session retained its choice	Start a new chat or explicitly switch it
Paid activity despite free main model	Aux, pinned jobs, subagents, external tools, or upstream fallbacks	Audit every billing surface in section 6

Privacy rules

• Free cloud inference sends prompts to third parties. Read OpenRouter's settings and each upstream provider's data policy before sending sensitive data.

• A cloud fallback can transmit prior conversation context, not just your last sentence.

• Local-only work requires local auxiliary routes and appropriately restricted tools too.

• Never share keys, `.env`, `auth.json`, private logs, or session databases.

• If a real key has already been pasted into chat or published, rotate/revoke it. Deleting the message is not enough.

• Keep normal command approvals enabled. “Free model” does not mean “safe to grant unlimited system access.”

17. Links and maintenance

Core setup

• Hermes home/downloads: https://hermes-agent.nousresearch.com/

• Source: https://github.com/NousResearch/hermes-agent

• Installation: https://hermes-agent.nousresearch.com/docs/getting-started/installation

• Models and auxiliary slots: https://hermes-agent.nousresearch.com/docs/user-guide/configuring-models

• Configuration: https://hermes-agent.nousresearch.com/docs/user-guide/configuration

• Fallbacks: https://hermes-agent.nousresearch.com/docs/user-guide/features/fallback-providers

• Local models: https://hermes-agent.nousresearch.com/docs/user-guide/local-models

• Profiles: https://hermes-agent.nousresearch.com/docs/user-guide/profiles

• Secrets: https://hermes-agent.nousresearch.com/docs/user-guide/secrets

• Security: https://hermes-agent.nousresearch.com/docs/user-guide/security

• Full documentation index: https://hermes-agent.nousresearch.com/docs/llms.txt

Free cloud

• OpenRouter keys: https://openrouter.ai/settings/keys

• Free models: https://openrouter.ai/models?pricing=free

• Public catalog: https://openrouter.ai/api/v1/models

• Free router: https://openrouter.ai/docs/guides/routing/routers/free-router

• Free variants: https://openrouter.ai/docs/guides/routing/model-variants/free

• Limits: https://openrouter.ai/docs/api/reference/limits

• Activity: https://openrouter.ai/activity

Optional local/automation/routing

• Ollama: https://ollama.com/download

• Hermes + Ollama: https://hermes-agent.nousresearch.com/docs/guides/local-ollama-setup

• Cron: https://hermes-agent.nousresearch.com/docs/user-guide/features/cron

• Delegation: https://hermes-agent.nousresearch.com/docs/user-guide/features/delegation

• TypeSafe: https://typesafe.ai/

• TypeSafe docs: https://docs.typesafe.ai/

• Community Jev MCP: https://github.com/codaaiteam/jev-mcp

Maintain it without turning it into a hobby

Recheck model availability and free pricing when a model stops working or before enabling a new automation. After Hermes upgrades, inspect fallback behavior and auxiliary settings. Review provider activity after significant changes. Avoid installing a proxy, swarm, or extra router until you can name the problem it solves.

The minimum useful setup is Hermes + one eligible Nous/OpenRouter free route + free-only fallbacks + explicit paid permission. Local inference and Jev are optional additions, not prerequisites.

18. NanoGPT (paid-last, crypto, no KYC)

Paid tier, but payable in crypto and without KYC — the acceptable last resort.

1. Open the referral for 5% off eligible usage: https://nano-gpt.com/r/AhJbKMxg (redeem code `AhJbKMxg`). No referral discount on provider-selection pricing or markup (Caching mode, pinned provider).

2. Create an API key in the NanoGPT dashboard and store it in your secret manager (`NANOGPT_API_KEY`), never in chat.

3. Add it to Hermes as a custom OpenAI-compatible provider; verify the current `base_url` and model IDs from the NanoGPT docs before wiring.

4. Only enable after the free routes are exhausted, and state the spend cap out loud first.

19. PPQ (paid-last, invite-only, privacy-first)

Invite-based access to strong paid models without email/phone signup.

1. Accept the invite: https://ppq.ai/invite/949880ca

2. Fund the account (check current payment methods and KYC terms on ppq.ai — terms change).

3. Create an API key, store it as `PPQ_API_KEY` in your secret manager.

4. Wire as a custom OpenAI-compatible provider in Hermes using PPQ's current `base_url`; confirm model IDs from their docs.

5. Treat as paid-last, same rule as NanoGPT.

20. CheaperInference (discounted paid aggregator)

One OpenAI-compatible API with discounted frontier models; fund from $5, usage-based, no monthly commitment. Use it instead of paying full price when a paid model is genuinely needed.

1. Sign up with the ref link: https://cheaperinference.com/?ref=XfdSBRMrj8 (or https://platform.cheaperinference.com/signup?ref=XfdSBRMrj8)

2. API base: `https://api.cheaperinference.com/v1` — store the key as `CHEAPERINFERENCE_API_KEY` in your secret manager.

3. Compare live discounts on the catalog before choosing a model (examples at time of writing: deepseek-v4.1-flash 55% off, claude-opus-5.5 35% off, glm-5.3-flash 42% off). Check https://platform.cheaperinference.com/docs.

4. Wire as a custom provider in Hermes; still paid — permission and spend cap first.

21. Kimi (cheap paid + referral credits)

Kimi (Moonshot AI) is a cheap strong-model route, and the referral program gives both sides membership-credit draws (3/7/15/30-day or 1-year).

1. Use the invite: https://www.kimi.ai/activities/en/invite/share?scenario=subscribe&from=share_poster&invitation_code=MJE8ZY (code `MJE8ZY`). Bind and subscribe, then enter the draw on the campaign page.

2. Get an API key (Moonshot platform), store it as `KIMI_API_KEY` in your secret manager.

3. Wire as a custom OpenAI-compatible provider (Moonshot endpoint) in Hermes, or via OpenRouter if the Kimi models are listed there.

4. Paid-last applies; referral credits reduce cost but are not a free tier.

Referral links above benefit the guide author. They never change the setup steps — free routes stay the default; chapters 18-21 are only for explicit, capped paid use.